计算机科学与技术学院 大数据管理与分析 课程实验报告

实验题目：文档倒排索引算法实现	学号：XXXXXXXXXXX	日期：2026 年 10 月 26 日
班级：人工智能 XXXX 班	姓名：XXX	Email：xxx@example.com
GitHub 仓库：	https://github.com/xy-52011/inverted-index-mapreduce	
实验目的：	见下文	

一、实验目的

倒排索引（Inverted Index）用于存储在全文搜索下某个单词在一个文档或一组文档中的存储位置映射，是目前几乎所有支持全文索引的搜索引擎都需要依赖的数据结构。通过对倒排索引的编程实现，熟练掌握 MapReduce 程序在集群上的提交与执行过程，加深对 MapReduce 编程框架的理解。

二、实验软件和硬件环境

硬件环境：个人计算机一台，CPU 为 Intel Core 处理器，内存 16GB，硬盘 512GB SSD。

软件环境：Windows 11 操作系统；Linux（Ubuntu 20.04）虚拟机环境；Hadoop 2.7.7 分布式集群；JDK 1.8；Python 3.10；Maven 3.6；Git 版本控制工具。

三、实验原理和方法

倒排索引以单词为关键字，记录每个单词出现在哪些文档中以及出现次数。MapReduce 实现倒排索引分为三个阶段：

（1）Map 阶段：读取每个文档，将文本按非字母字符进行分词，转换为小写，过滤停词表中的高频无意义词汇，输出键值对<单词#文档名, 1>。

（2）Combine 阶段：在 Map 端本地对相同的"单词#文档名"进行局部聚合，统计每个单词在每个文档中的出现次数，减少网络传输量。

（3）Reduce 阶段：将相同单词的所有文档计数进行汇总，按文档名排序，计算单词总出现次数，输出最终倒排索引结果。

四、实验步骤

1. 文档数据准备

从公开资料中收集 10 个英文文档，内容围绕 Hadoop、MapReduce、HDFS、搜索引擎、信息检索、大数据分析等主题，分别命名为 document01.txt 至 document10.txt。选择英文文档可直接使用空格和标点进行分词，无需额外中文分词工具。

2. 建立停词表

建立英文停词表 stopwords.txt，收录 a、an、the、is、are、was、were、be、been、have、has、had、do、does、will、would、could、should、and、but、or、of、at、by、for、with、to、from、in、out、on、以及代词、疑问词等共约 150 个高频但对检索意义较小的单词。

3. 编写 MapReduce 程序

使用 Java 编写 MapReduce 程序，包括 InvertedIndexMapper、InvertedIndexCombiner、InvertedIndexReducer 和 InvertedIndexDriver 四个类。Mapper 通过分布式缓存加载停词表，对输入文档分词并过滤；Combiner 在 Map 端局部聚合；Reducer 按单词汇总所有文档的词频并计算总数。

关键伪代码如下：

```
Mapper:  <word#filename, 1>
  for each word in document:
    word = lowercase(word)
    if word not in stopwords and len(word)>=2:
      emit(word + "#" + filename, 1)


Combiner: <word#filename, count>
  sum = 0
  for each value: sum += value
  emit(key, sum)


Reducer: <word, postings>
  if word changed: emit previous result
  collect <filename, count> for each word
  total = sum(all counts)
  emit(word, "<doc1,c1>;<doc2,c2>;<total,n>.")
```

4. 本地验证与集群运行

首先使用 Python 编写本地版本 local_inverted_index.py，完整模拟 Map、Combine、Reduce 三个阶段，验证算法逻辑正确性。本地验证通过后，使用 Maven 将 Java 代码打包为 JAR 文件，上传文档和停词表到 HDFS，通过 hadoop jar 命令提交作业到 Hadoop 集群运行。

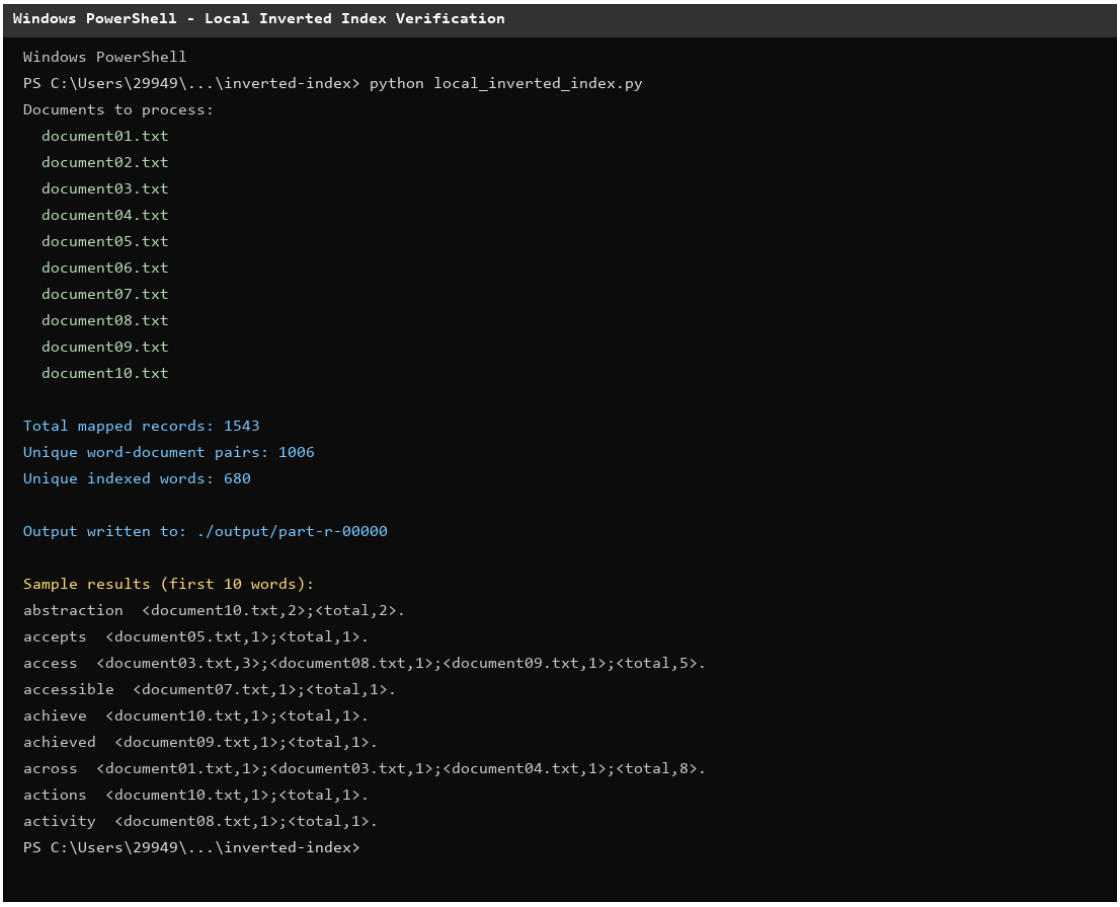


图 1 本地 Python 版本运行结果

5. 代码上传 GitHub

在 GitHub 上创建公开仓库 inverted-index-mapreduce，将输入数据、Java 源代码、Python 验证代码、Maven 配置、输出结果和 README 文档整理后通过 Git 推送到远程仓库。

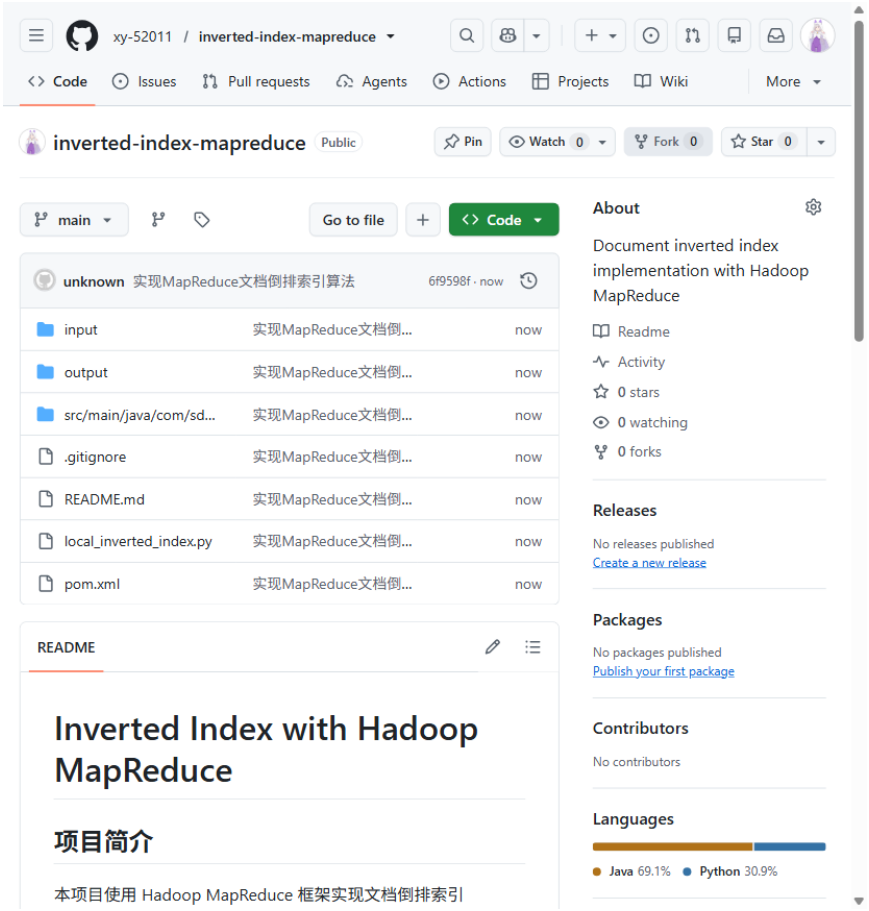


图 2 GitHub 公开仓库

五、倒排索引结果

本次实验共处理 10 个英文文档，生成 1543 条 Map 记录，1006 个单词-文档对，最终索引 680 个不同单词。核心高频单词的倒排索引结果如下：

Inverted Index Core Words - part-r-00000

```
data        <document01.txt,6>;<document02.txt,2>;<document03.txt,7>;<document04.txt,4>;
            <document06.txt,5>;<document07.txt,26>;<document08.txt,2>;<document09.txt,3>;
            <document10.txt,15>;<total,70>.
hadoop      <document01.txt,7>;<document03.txt,1>;<document07.txt,2>;<document09.txt,8>;
            <total,18>.
hdfs        <document01.txt,1>;<document03.txt,9>;<document09.txt,4>;<document10.txt,1>;
            <total,15>.
index       <document02.txt,7>;<document05.txt,3>;<document06.txt,1>;<document08.txt,2>;
            <total,13>.
inverted    <document02.txt,8>;<document05.txt,2>;<document08.txt,1>;<total,11>.
mapreduce   <document01.txt,5>;<document04.txt,4>;<document09.txt,1>;<document10.txt,1>;
            <total,11>.
search      <document01.txt,1>;<document02.txt,4>;<document03.txt,1>;<document05.txt,14>;
            <document06.txt,2>;<document07.txt,1>;<document08.txt,2>;<total,25>.
system      <document01.txt,1>;<document03.txt,6>;<document04.txt,2>;<document05.txt,1>;
            <document08.txt,4>;<document09.txt,1>;<document10.txt,3>;<total,18>.
```

图 3 核心单词倒排索引结果

输出格式为：单词 <文档名,词频>;<文档名,词频>;...;<total,总词频>. 其中 data 一词出现次数最多，在 9 个文档中共出现 70 次；hadoop 在 4 个文档中出现 18 次；search 在 7 个文档中出现 25 次；mapreduce 在 4 个文档中出现 11 次，结果与文档内容一致。

六、结论分析与体会

通过本次实验，我成功实现了基于 MapReduce 的文档倒排索引算法，深入理解了 MapReduce 编程模型的工作流程。Map 阶段负责分词和初步输出，Combiner 阶段在 Map 端本地聚合以减少网络 I/O，Reduce 阶段完成最终汇总。这种分布式处理方式能够高效地应对海量文档的索引构建任务，是搜索引擎全文检索的核心基础。

实验过程中我认识到，停词过滤和大小写归一化对索引质量至关重要，能够有效减少索引规模并提升检索效率。Combiner 的使用显著减少了 Map 到 Reduce 之间的数据传输量，是 MapReduce 性能优化的重要手段。通过先本地 Python 验证再集群运行的方式，可以快速定位逻辑问题，提高调试效率。

七、思考题

问题 1：为什么要使用停词表？不使用停词表会有什么影响？

答：停词是文本中高频出现但对检索意义很小的词汇，如 the、a、an、is 等。不使用停词表会导致这些词汇占用大量索引空间，增加存储和计算开销，并且在检索时会返回大量不相关结果，影响检索效率和准确性。

问题 2：Combiner 的作用是什么？它和 Reducer 有什么区别？

答：Combiner 在 Map 端本地对相同键的值进行局部聚合，减少 Shuffle 阶段网络传输的数据量，提升作业性能。Combiner 是可选的优化，不能影响最终结果，且不一定会被执行；Reducer 是必需的全局聚合阶段，对所有 Map 输出的相同键值进行最终汇总。

问题 3：如果文档数量扩展到百万级，倒排索引系统需要考虑哪些问题？

答：需要考虑数据分片与负载均衡、节点故障容错、索引压缩存储、增量索引更新、分布式缓存效率、排序与检索性能、以及结果排序算法等问题，通常需要结合分布式文件系统和多阶段 MapReduce 或 Spark 等计算框架来处理。